Northern University of Business and Technology Khulna
Department of Computer Science and Engineering

WEEK 12: API DOCUMENTATION

Project Title: AI-Powered Resume Analyzer and Career Copilot (RESUMIND)

Course Title: Mobile Computing Lab

Course Code: CSE4204

Submitted to:
Md. Riaz Mahmud
Assistant Professor
Department of Computer Science and Engineering
Northern University of Business and Technology, Khulna

Submission Date: 22 October 2026

TEAM INFORMATION SHEET

Field	Details
Official Team Name	CSE4204-8A-T05
Section	8A
Project Title	AI-Powered Resume Analyzer and Career Copilot (RESUMIND)
Team Member List with IDs	1. Mahammad Hasan (ID: 11220320828) - Team Leader / Frontend Developer + UI/UX Designer 2. Shanawaz Sakib (ID: 11220320916) - Core System Integration Engineer 3. Ashikur Rahman Himel (ID: 11220320833) - Database Storage & Quality Controller 4. Rasel Ratul (ID: 11220320827) - Backend Developer
Live Application Link	https://ai-resume-analyzer-psi-lime.vercel.app
GitHub Repository Link	https://github.com/mahamud894/AI_Resume_Analyzer
Figma Link	https://www.figma.com/design/Ls0B3DIjxRKrP2RIzYQxPG/AI-Resume-Analyzer?node-id=2-2&t=lFEVQCoUHt0JSumI-1

1. Overview

The RESUMIND backend is a JSON REST API built with Node.js and Express 5. The React frontend uses it for authentication, resume storage and all AI features. This document lists every endpoint with its **purpose**, **input** and **output**, followed by error codes, rate limits and the data models.

Table 1: Base URLs

Environment	Base URL
Production (Render)	https://resumind-api-3kpd.onrender.com/api
Local development	http://localhost:5000/api
Frontend that calls it	https://ai-resume-analyzer-psi-lime.vercel.app

1.1 Conventions

- Requests and responses use `application/json`, except file upload (`multipart/form-data`) and file download (binary).
- Ids are MongoDB ObjectIds (24 hex characters). An invalid id returns `400`.
- Dates are ISO-8601 strings in UTC, e.g. `2026-10-10T10:35:55.065Z`.
- Every resource belongs to one user. Requesting another user's resource returns `404` (not `403`), so ids cannot be probed.

1.2 Authentication

Endpoints marked 🔒 require a JSON Web Token in the `Authorization` header. The token is returned by register, login, Google sign-in and password change. It is valid for 7 days and becomes invalid when the password changes.

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

1.3 Errors

All errors share one JSON shape. Validation errors also list the failing fields.

```
HTTP 400
{ "message": "Job title is required (max 100 characters)",
  "details": [ { "field": "jobTitle", "message": "Job title is required (max 100
characters)" } ] }
```

Table 2: HTTP status codes

Code	Meaning in this API
200 / 201	Success / resource created
400	Invalid input, invalid id, or wrong current password
401	Missing, invalid or expired token; wrong login
404	Resource not found or not owned by the user; unknown route
409	E-mail or username already registered
413	Upload or edited CV too large
422	PDF contains no readable text (scanned image)
429	Rate limit reached (see 1.4)
502	Gemini returned an error or invalid output
503	AI key / Google client id not configured, or Gemini still busy after 3 retries

1.4 Rate limits

Limiter	Applies to	Limit
auth	register, login, google, change password, delete account	20 requests per 15 minutes per IP
ai	analyze, improve, improved/score, cover-letter (POST)	15 requests per hour per user (failed requests do not count)

Responses include the standard `RateLimit` and `RateLimit-Policy` headers (IETF draft-8), so the client can see the remaining quota.

2. Endpoint Summary

Table 3: All endpoints

Method	Route	Auth	Purpose
GET	`/health`		Check that the API is running (used by Render's health check)
POST	`/auth/register`		Create a new account and log the user in
POST	`/auth/login`		Log in with e-mail and password
POST	`/auth/google`		Sign in or sign up with Google
GET	`/auth/me`	🔒	Return the logged-in user (used when the app starts)

Method	Route	Auth	Purpose
PUT	`/auth/password`	🔒	Change the password (or set a first password for a Google-only account)
DELETE	`/auth/me`	🔒	Delete the account and everything it owns (resumes, analyses, improved resumes, cover letters, files)
POST	`/resume/upload`	🔒	Upload a resume PDF and an optional page-1 preview image
GET	`/resume`	🔒	List the user's resumes (newest first), each with its analyses and improved-resume summary — used by the dashboard
GET	`/resume/:id`	🔒	Get one resume with its analyses
GET	`/resume/:id/file`	🔒	Download the original PDF
GET	`/resume/:id/image`	🔒	Download the page-1 preview image
DELETE	`/resume/:id`	🔒	Delete a resume with all its analyses, improved resumes, cover letters and both files
POST	`/resume/analyze`	🔒	Analyse a resume against a job with Gemini: overall score plus ATS, tone & style, content, structure and skills scores with tips
POST	`/resume/analysis/:analysisId/improve`	🔒	Rewrite the resume as a Europass CV that fixes every "improve" tip without inventing facts, and list each change
GET	`/resume/analysis/:analysisId/improved`	🔒	Get the saved improved resume (and a `coverLetter` summary if a letter exists)
PUT	`/resume/analysis/:analysisId/improved`	🔒	Save the user's inline edits of the CV
POST	`/resume/analysis/:analysisId/improved/score`	🔒	Score the improved CV (edited version if saved) for the same job, with placeholders removed
POST	`/resume/analysis/:analysisId/cover-letter`	🔒	Generate a one-page European business letter from the improved CV and the job
GET	`/resume/analysis/:analysisId/cover-letter`	🔒	Get the saved cover letter
PUT	`/resume/analysis/:analysisId/cover-letter`	🔒	Save the user's edited letter text

3. Health Endpoints

3.1 GET /api/health

Purpose	Check that the API is running (used by Render's health check).
Method · Route	GET /api/health
Authentication	Not required
Input	None
Output	200 `{ status: "ok", time }`
Errors	—

4. Authentication Endpoints

4.1 POST /api/auth/register

Purpose	Create a new account and log the user in.
Method · Route	POST /api/auth/register
Authentication	Not required
Input	JSON body: `username` — 3–30 chars, letters/numbers/_/. , unique `email` — valid e-mail, unique (stored lowercase) `password` — 8–72 chars (bcrypt-hashed)
Output	201 `{ token, user: { id, username, email, hasPassword, createdAt } }`
Errors	400 invalid input · 409 e-mail or username already used · 429 too many attempts
Rate limit	Auth limiter — 20 per 15 min per IP

Example

```
POST /api/auth/register
{ "username": "tanveer", "email": "tanveer@example.com", "password": "secret123" }

201 Created
{ "token": "eyJhbGciOiJIUzI1NiIs...",
  "user": { "id": "6aca150b1f91ced92f7ef8c0", "username": "tanveer",
            "email": "tanveer@example.com", "createdAt": "2026-10-
10T10:35:55.065Z" } }
```

4.2 POST /api/auth/login

Purpose	Log in with e-mail and password.
Method · Route	POST /api/auth/login
Authentication	Not required
Input	JSON body: `email`, `password`
Output	200 `{ token, user }` (same shape as register)
Errors	400 invalid input · 401 "Invalid email or password" · 429 too many attempts
Rate limit	Auth limiter — 20 per 15 min per IP

4.3 POST /api/auth/google

Purpose	Sign in or sign up with Google. The backend verifies the Google ID token and links it to an existing account with the same verified e-mail.
Method · Route	POST /api/auth/google
Authentication	Not required
Input	JSON body: `credential` — Google ID token from Google Identity Services
Output	201 new user / 200 existing user `{ token, user }`
Errors	400 missing credential · 401 invalid token or unverified e-mail · 503 GOOGLE_CLIENT_ID not configured
Rate limit	Auth limiter — 20 per 15 min per IP

4.4 GET /api/auth/me

Purpose	Return the logged-in user (used when the app starts).
Method · Route	GET /api/auth/me
Authentication	Required — Bearer JWT 🔒
Input	None
Output	200 `{ user }`
Errors	401 missing / invalid / expired token

4.5 PUT /api/auth/password

Purpose	Change the password (or set a first password for a Google-only account). All older tokens stop working, so other devices are logged out.
Method · Route	PUT /api/auth/password

Authentication	Required — Bearer JWT 🔒
Input	JSON body: `currentPassword` (not needed for Google-only accounts), `newPassword` 8–72 chars
Output	200 `{ token, user }` — the client stores the new token
Errors	400 wrong current password or invalid new password · 401
Rate limit	Auth limiter — 20 per 15 min per IP

4.6 DELETE /api/auth/me

Purpose	Delete the account and everything it owns (resumes, analyses, improved resumes, cover letters, files).
Method · Route	DELETE /api/auth/me
Authentication	Required — Bearer JWT 🔒
Input	JSON body: `password` (Google-only accounts: `confirmEmail`)
Output	200 `{ message }`
Errors	400 wrong password / e-mail · 401
Rate limit	Auth limiter — 20 per 15 min per IP

5. Resumes Endpoints

5.1 POST /api/resume/upload

Purpose	Upload a resume PDF and an optional page-1 preview image. Files are stored in MongoDB GridFS.
Method · Route	POST /api/resume/upload
Authentication	Required — Bearer JWT 🔒
Input	multipart/form-data: `resume` — required, PDF, must start with %PDF-, max 10 MB `image` — optional PNG/JPEG preview
Output	201 `{ resume: { id, userId, originalName, fileSize, uploadDate, fileUrl, imageUrl } }`
Errors	400 no file / not a PDF / wrong field · 401 · 413 file too large

Example

```
201 Created
{ "resume": { "id": "6aca15131f91ced92f7ef8c2", "userId": "6aca150b1f91ced92f7ef8c0",
    "originalName": "my resume.pdf", "fileSize": 84213, "uploadDate": "2026-10-
10T10:36:03.089Z",
    "fileUrl": "/api/resume/6aca15131f91ced92f7ef8c2/file",
    "imageUrl": "/api/resume/6aca15131f91ced92f7ef8c2/image" } }
```

5.2 GET /api/resume

Purpose	List the user's resumes (newest first), each with its analyses and improved-resume summary — used by the dashboard.
Method · Route	GET /api/resume
Authentication	Required — Bearer JWT 🔒
Input	None
Output	200 `{ resumes: [ { …resume, analyses: [ { …analysis, improved: { id, afterFeedback, scoredAt } } ] } ] }`
Errors	401

5.3 GET /api/resume/:id

Purpose	Get one resume with its analyses.
Method · Route	GET /api/resume/:id
Authentication	Required — Bearer JWT 🔒
Input	URL: `id` — resume ObjectId
Output	200 `{ resume }`
Errors	400 invalid id · 401 · 404 not found / not yours

5.4 GET /api/resume/:id/file

Purpose	Download the original PDF.
Method · Route	GET /api/resume/:id/file
Authentication	Required — Bearer JWT 🔒
Input	URL: `id`
Output	200 PDF bytes (`application/pdf`)
Errors	400 · 401 · 404

5.5 GET /api/resume/:id/image

Purpose	Download the page-1 preview image.
Method · Route	GET /api/resume/:id/image
Authentication	Required — Bearer JWT 🔒
Input	URL: `id`
Output	200 PNG/JPEG bytes
Errors	400 · 401 · 404

5.6 DELETE /api/resume/:id

Purpose	Delete a resume with all its analyses, improved resumes, cover letters and both files.
Method · Route	DELETE /api/resume/:id
Authentication	Required — Bearer JWT 🔒
Input	URL: `id`
Output	200 `{ message: "Resume deleted", id }`
Errors	400 · 401 · 404

6. AI features Endpoints

6.1 POST /api/resume/analyze

Purpose	Analyse a resume against a job with Gemini: overall score plus ATS, tone & style, content, structure and skills scores with tips. Takes about 10–20 s.
Method · Route	POST /api/resume/analyze
Authentication	Required — Bearer JWT 🔒
Input	JSON body: `resumeId` — one of your resumes `companyName` — optional, ≤ 100 chars `jobTitle` — required, ≤ 100 chars `jobDescription` — required, 20–10 000 chars
Output	201 `{ analysis: { id, resumeId, userId, companyName, jobTitle, jobDescription, feedback, createdAt } }`
Errors	400 invalid input · 404 resume not found · 422 PDF has no readable text · 429 rate limit · 502 Gemini failed · 503 key missing or model busy after 3 retries
Rate limit	AI limiter — 15 per hour per user

Example

```
POST /api/resume/analyze
{ "resumeId": "6aca15131f91ced92f7ef8c2", "companyName": "Google",
  "jobTitle": "Junior Frontend Developer", "jobDescription": "We need React +
TypeScript..." }

201 Created  → analysis.feedback:
{ "overallScore": 72,
  "ATS":          { "score": 78, "tips": [ { "type": "good", "tip": "Clear section
headings" } ] },
  "toneAndStyle": { "score": 75, "tips": [ { "type": "improve", "tip": "Avoid first
person",
                                                "explanation": "..." } ] },
  "content": { "score": 65, "tips": [...] }, "structure": { "score": 80, "tips": [...]
},
  "skills":  { "score": 70, "tips": [...] } }
```

6.2 POST /api/resume/analysis/:analysisId/improve

Purpose	Rewrite the resume as a Europass CV that fixes every "improve" tip without inventing facts, and list each change. Regenerating replaces the previous version. Takes about 20–40 s.
Method · Route	POST /api/resume/analysis/:analysisId/improve
Authentication	Required — Bearer JWT 🔒
Input	URL: `analysisId` — one of your analyses. No body.
Output	201 `{ improved: { id, analysisId, resumeId, userId, europassCv, changes[], createdAt } }`
Errors	400 invalid id · 401 · 404 analysis not found · 422 · 429 · 502 · 503
Rate limit	AI limiter — 15 per hour per user

Example

```
201 Created (shortened)
{ "improved": {
    "europassCv": {
      "personalInfo": { "fullName": "Demo Student", "email": "demo@resumind.dev",
                        "phone": "[Add your phone number]", "address": "Khulna,
Bangladesh" },
      "aboutMe": "Junior Frontend Developer with hands-on experience in React...",
      "workExperience": [ { "startDate": "[Add start date MM/YYYY]", "jobTitle":
"Frontend Intern", ... } ],
      "education": [ ... ], "languageSkills": { ... }, "digitalSkills": [ "React",
"TypeScript" ], ... },
    "changes": [ { "section": "About Me", "category": "toneAndStyle", "before":
"(missing)",
                   "after": "Junior Frontend Developer...", "reason": "Added a
summary..." } ] } }
```

6.3 GET /api/resume/analysis/:analysisId/improved

Purpose	Get the saved improved resume (and a `coverLetter` summary if a letter exists).
Method · Route	GET /api/resume/analysis/:analysisId/improved
Authentication	Required — Bearer JWT 🔒
Input	URL: `analysisId`
Output	200 `{ improved }`
Errors	401 · 404 "No improved resume yet for this analysis"

6.4 PUT /api/resume/analysis/:analysisId/improved

Purpose	Save the user's inline edits of the CV. The AI version is kept; edits are stored in `editedCv`. Sending `null` resets to the AI version.
Method · Route	PUT /api/resume/analysis/:analysisId/improved
Authentication	Required — Bearer JWT 🔒
Input	JSON body: `europassCv` — whole CV object (≤ 100 KB, unknown fields dropped) or `null`
Output	200 `{ improved }` with `editedCv` and `editedAt`
Errors	400 not an object · 404 · 413 larger than 100 KB

6.5 POST /api/resume/analysis/:analysisId/improved/score

Purpose	Score the improved CV (edited version if saved) for the same job, with placeholders removed. Used for the before/after comparison and the score-history chart.
Method · Route	POST /api/resume/analysis/:analysisId/improved/score
Authentication	Required — Bearer JWT 🔒
Input	URL: `analysisId`. No body.
Output	200 `{ improved }` with `afterFeedback` (same shape as `feedback`) and `scoredAt`
Errors	401 · 404 no improved resume · 429 · 502 · 503
Rate limit	AI limiter — 15 per hour per user

6.6 POST /api/resume/analysis/:analysisId/cover-letter

Purpose	Generate a one-page European business letter from the improved CV and the job. Gemini writes the opening, two body paragraphs, closing and highlights; the backend adds the fixed parts. Body length is kept at 250–400 words.
Method · Route	POST /api/resume/analysis/:analysisId/cover-letter
Authentication	Required — Bearer JWT 🔒
Input	JSON body (all optional): `hiringManagerName` ≤ 100 chars `companyAddress` ≤ 300 chars `jobSource` ≤ 100 chars (e.g. LinkedIn) `tone` — "formal" (default) or "modern"
Output	201 `{ coverLetter: { id, improvedResumeId, analysisId, inputs, letter: { sender, date, recipient, subject, salutation, opening, body[2], closing, signOff, signatureName, wordCount, highlights[] }, editedText } }`
Errors	400 invalid input / no improved resume yet · 401 · 404 · 429 · 502 · 503
Rate limit	AI limiter — 15 per hour per user

Example

```
POST /api/resume/analysis/6acb02ce79dbe54bfe24a21b/cover-letter
{ "hiringManagerName": "Ms. Sarah Rahman", "jobSource": "LinkedIn", "tone": "formal" }

201 Created → coverLetter.letter (shortened)
{ "subject": "Application for Junior Frontend Developer Position",
  "salutation": "Dear Ms. Rahman,", "opening": "Discovering the Junior Frontend
Developer opening...",
  "body": [ "...", "..." ], "closing": "...", "signOff": "Yours sincerely,",
"wordCount": 304 }
```

6.7 GET /api/resume/analysis/:analysisId/cover-letter

Purpose	Get the saved cover letter.
Method · Route	GET /api/resume/analysis/:analysisId/cover-letter
Authentication	Required — Bearer JWT 🔒
Input	URL: `analysisId`
Output	200 `{ coverLetter }`
Errors	401 · 404 "No cover letter yet for this analysis"

6.8 PUT /api/resume/analysis/:analysisId/cover-letter

Purpose	Save the user's edited letter text. An empty string resets to the generated letter.
Method · Route	PUT /api/resume/analysis/:analysisId/cover-letter
Authentication	Required — Bearer JWT 🔒
Input	JSON body: `editedText` — plain text, paragraphs separated by blank lines, ≤ 8000 chars
Output	200 `{ coverLetter }` with `editedText`
Errors	400 too long / not text · 404 no letter yet

6. Data Models

Responses return documents with an `id` field instead of MongoDB's `_id`; password hashes and internal fields are never returned.

Table 4: Response objects

Object	Fields
`user`	id, username, email, hasPassword, createdAt
`resume`	id, userId, originalName, fileSize, uploadDate, fileUrl, imageUrl, analyses[]
`analysis`	id, resumeId, userId, companyName, jobTitle, jobDescription, feedback, createdAt, improved?
`feedback`	overallScore (0–100); ATS, toneAndStyle, content, structure, skills — each { score, tips: [{ type: good | improve, tip, explanation? }] }
`improved`	id, analysisId, resumeId, userId, europassCv, editedCv, editedAt, changes[], afterFeedback, scoredAt, coverLetter?
`europassCv`	personalInfo, aboutMe, workExperience[], education[], languageSkills, digitalSkills[], projects[], certifications[], communicationSkills, organisationalSkills, volunteering[], references
`change`	section, category, before, after, reason
`coverLetter`	id, improvedResumeId, analysisId, userId, inputs, letter, editedText, createdAt, updatedAt

7. Testing the API

7.1 With curl

```
# 1. register and copy the token
curl -X POST https://resumind-api-3kpd.onrender.com/api/auth/register -H "Content-
Type: application/json" \
     -d '{"username":"tester","email":"tester@example.com","password":"secret123"}'
TOKEN=eyJhbGciOi...

# 2. upload a resume
curl -X POST https://resumind-api-3kpd.onrender.com/api/resume/upload -H
"Authorization: Bearer $TOKEN" \
     -F "resume=@my-resume.pdf;type=application/pdf"

# 3. analyse it for a job
curl -X POST https://resumind-api-3kpd.onrender.com/api/resume/analyze -H
"Authorization: Bearer $TOKEN" \
     -H "Content-Type: application/json" \
     -d '{"resumeId":"<id>","jobTitle":"Junior Frontend
Developer","jobDescription":"React, TypeScript, REST APIs ..."}'
```

7.2 Automated tests

`cd backend && npm test` runs 48 Vitest + Supertest tests against an in-memory MongoDB with Gemini mocked: auth (10), Google sign-in (9), resumes (12), analysis (4) and Europass / score / cover letter (13). They run automatically in GitHub Actions on every push.